Aula 10 – Ordenação

Algortimos Simples- Bubble Sort, Straight Sort, Insertion Sort

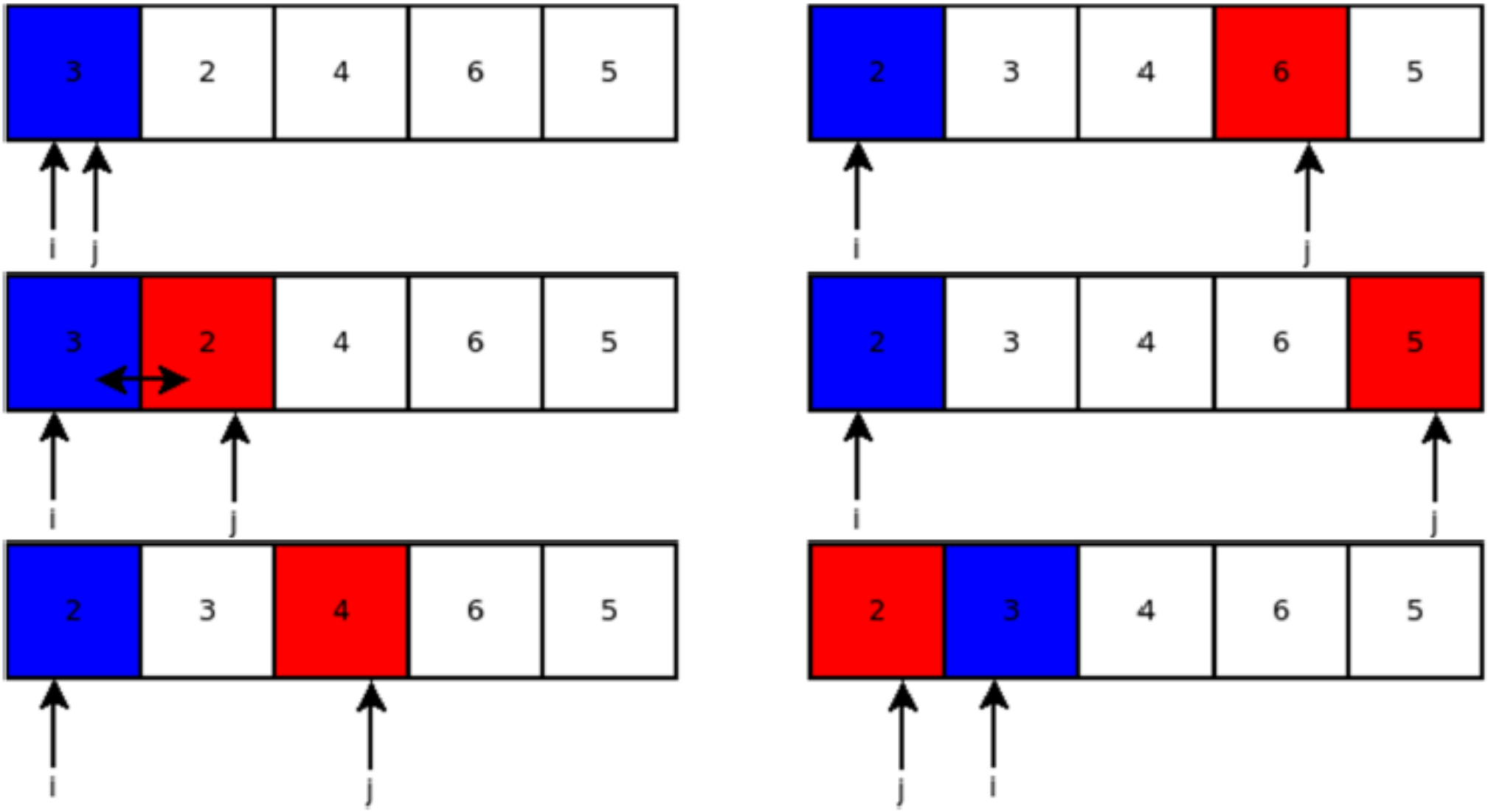
Algortimos Intermédios – Shell Sort

Algortimos Avançados – Merge Sort

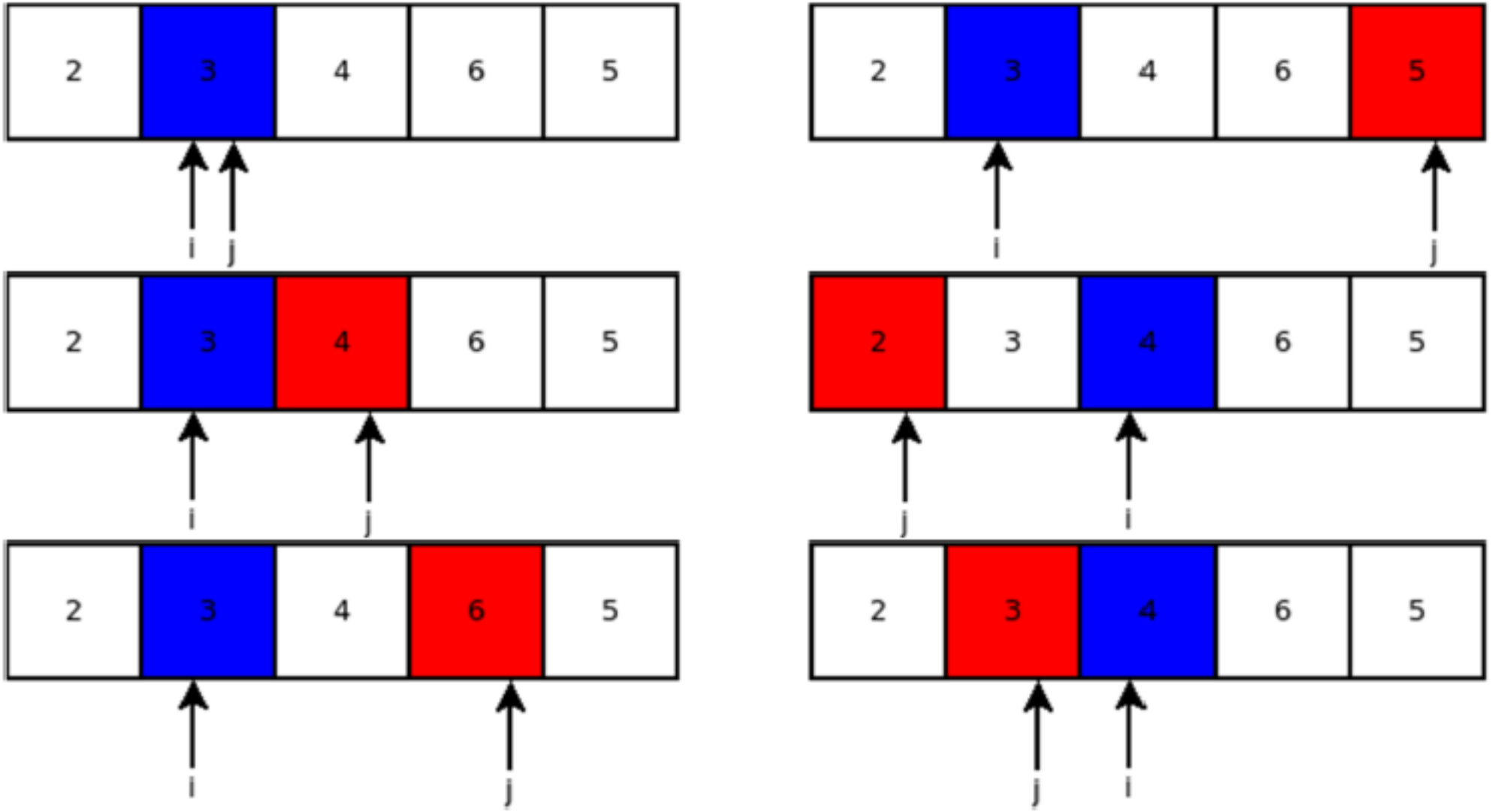
Bubble Sort

```
Para i de 0 até lista.tamanho -1
    Para j de 0 até lista.tamanho -1
        Se o lista[i] < lista[j]
            Trocar(lista[i], lista[j])
```

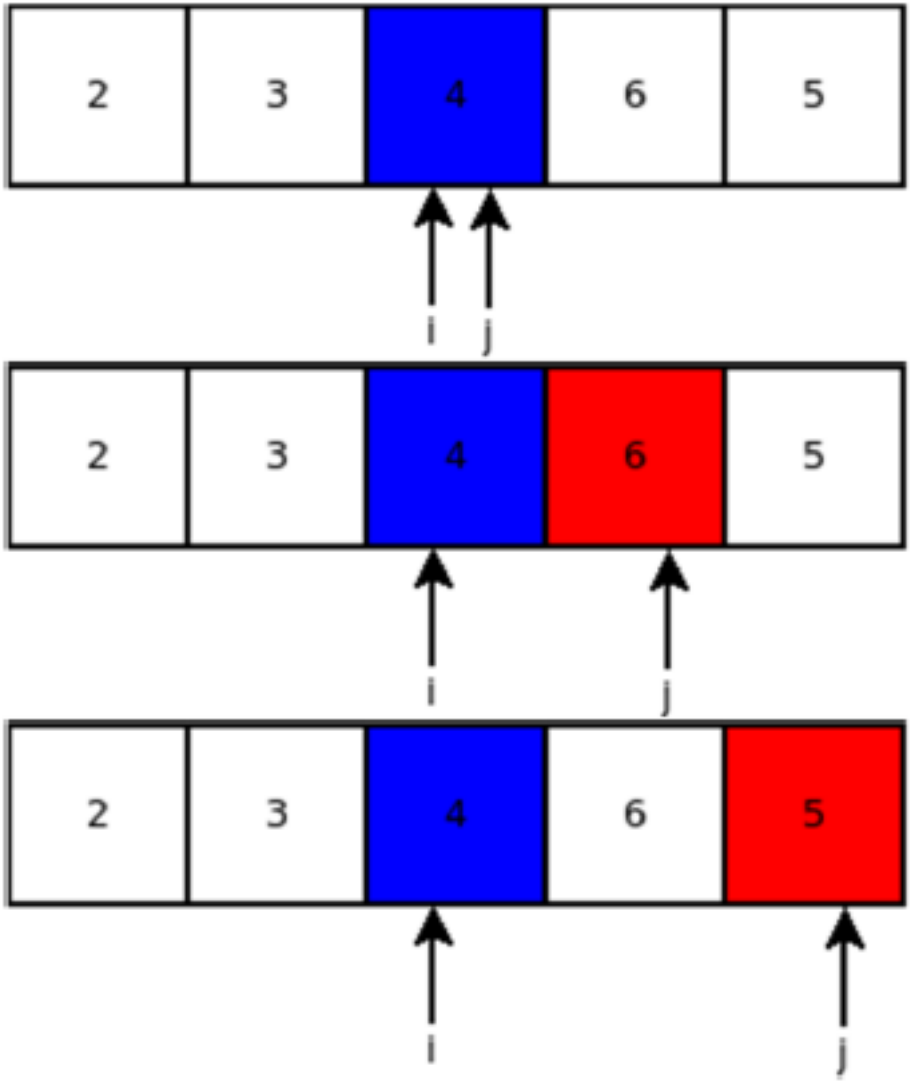
Bubble Sort

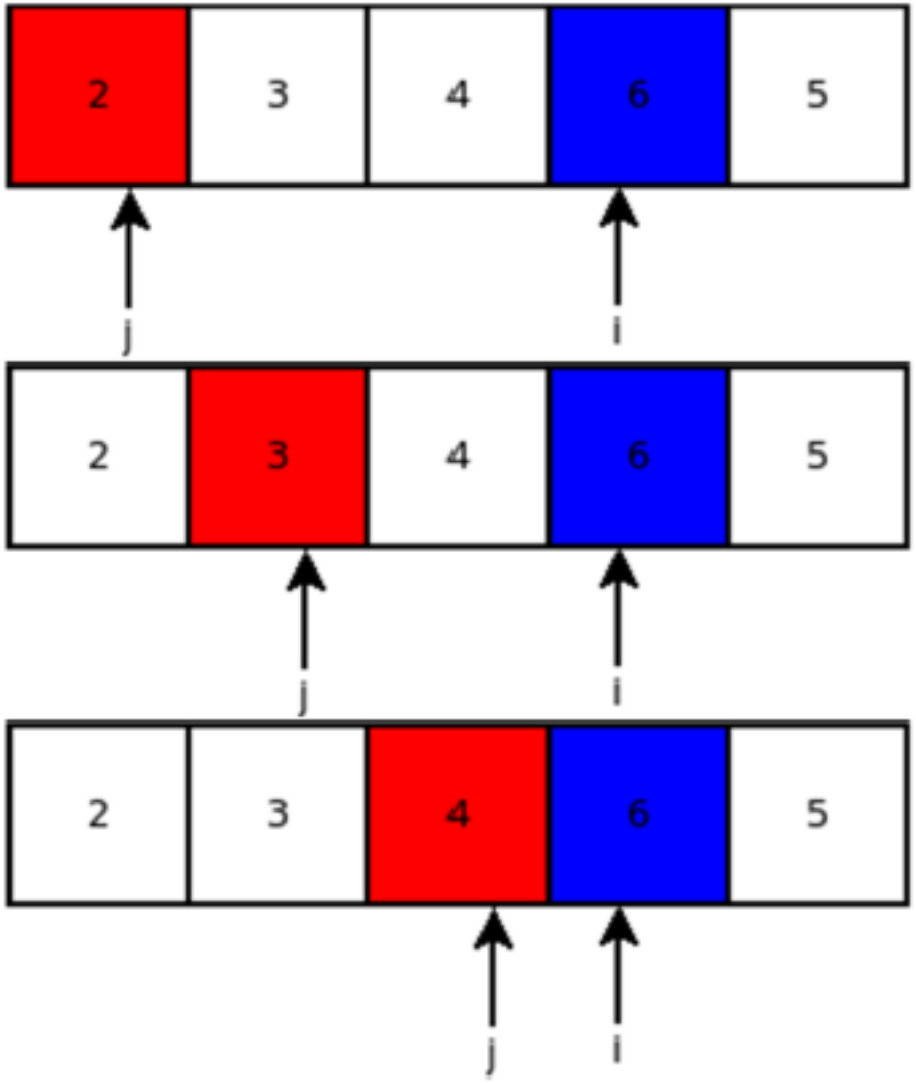


Bubble Sort

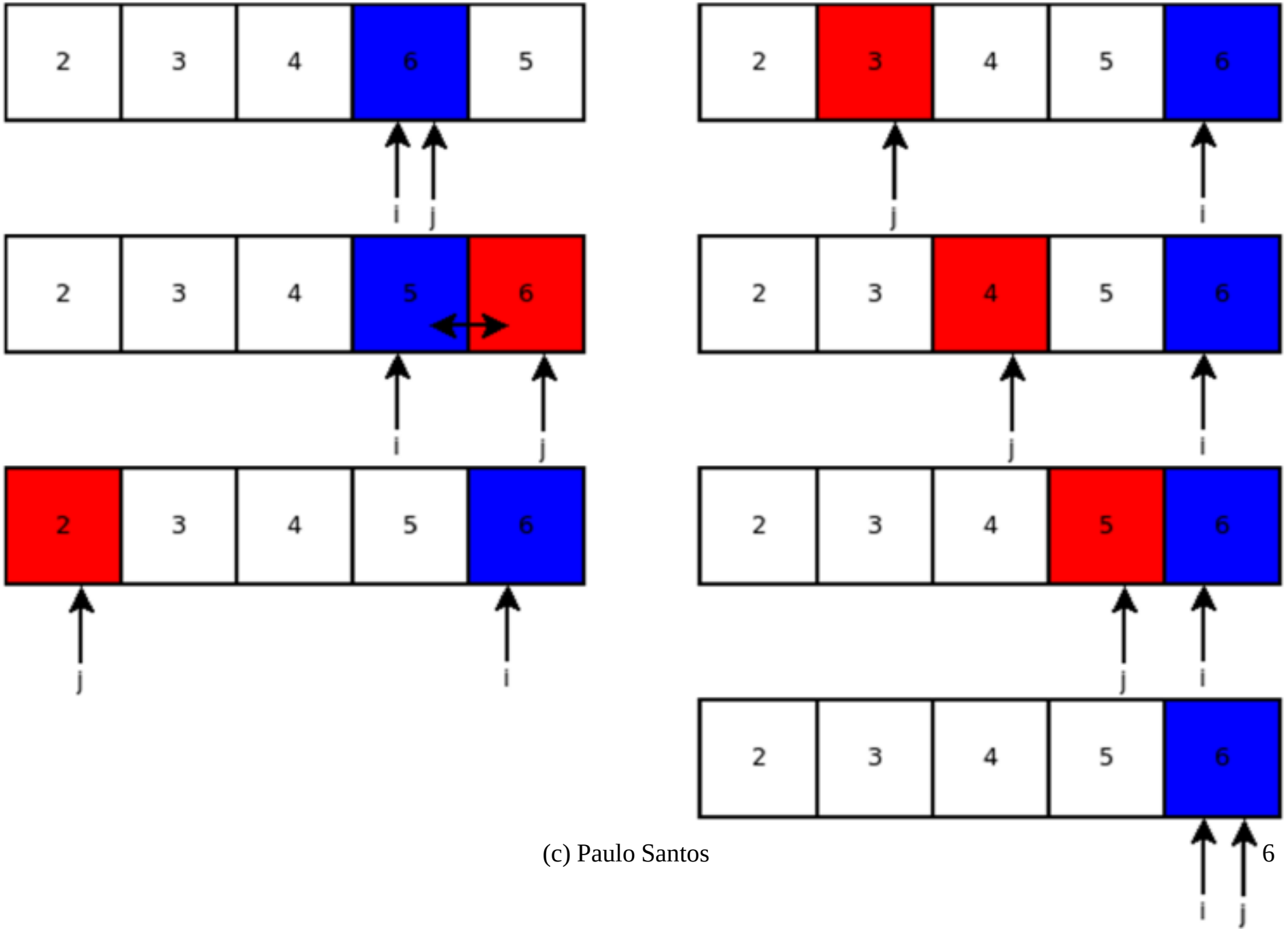


Bubble Sort





Bubble Sort



Análise de Bubble

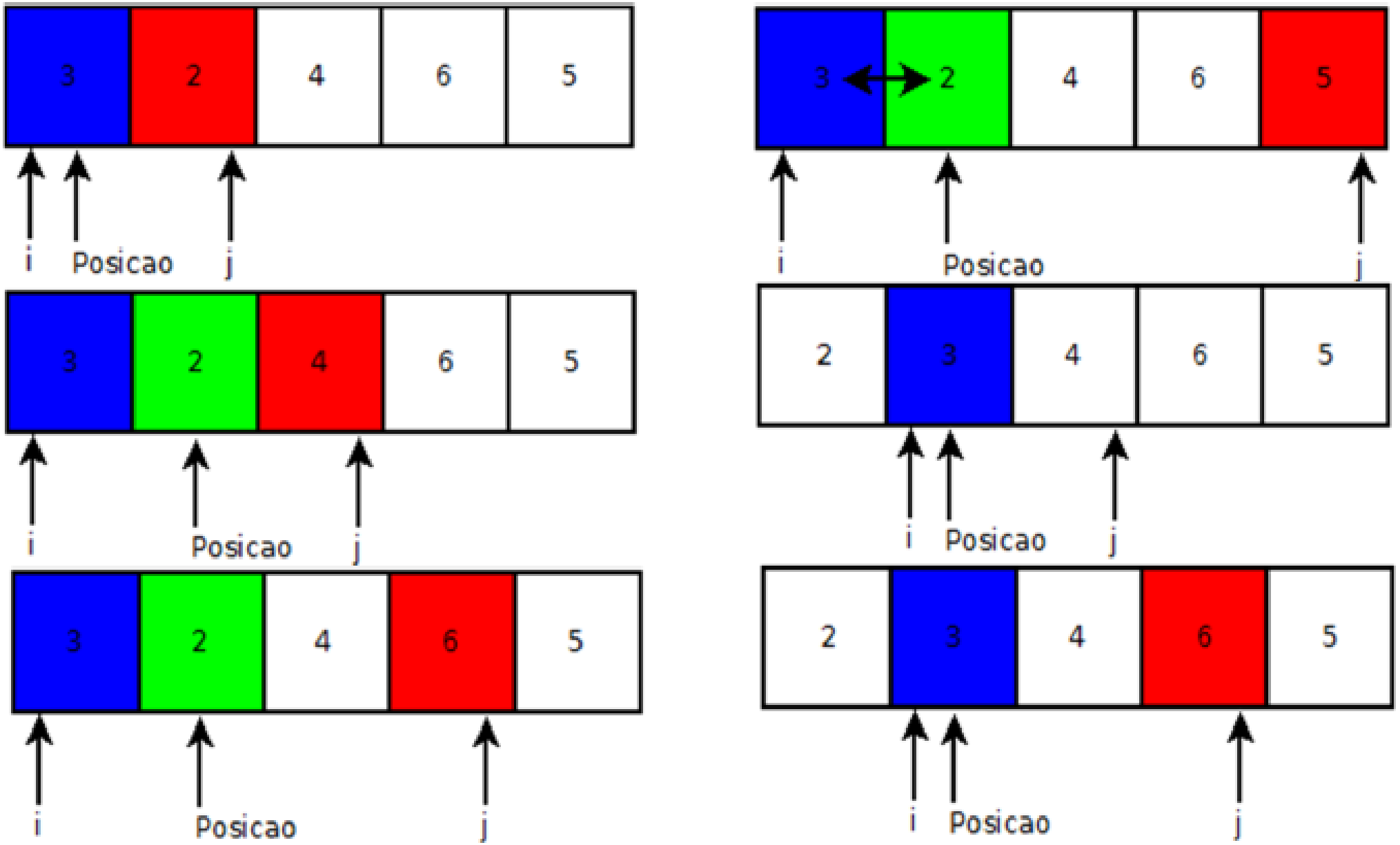
- Temporal
- Pior, melhor e caso médio
- $O(n^2)$ - quadrático
- Espacial
- S(1) – constante

- Melhoramentos:
- O segundo ciclo começar na posição i

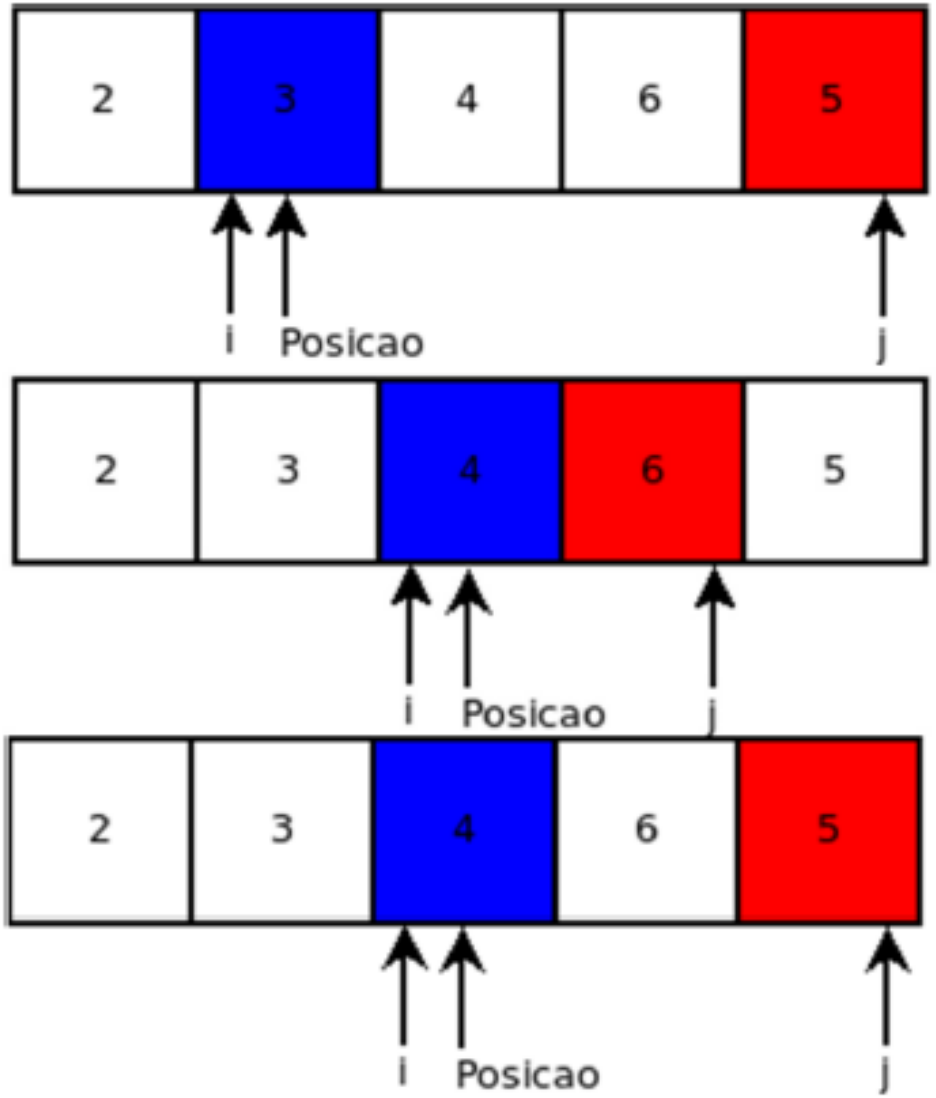
Straight Sort

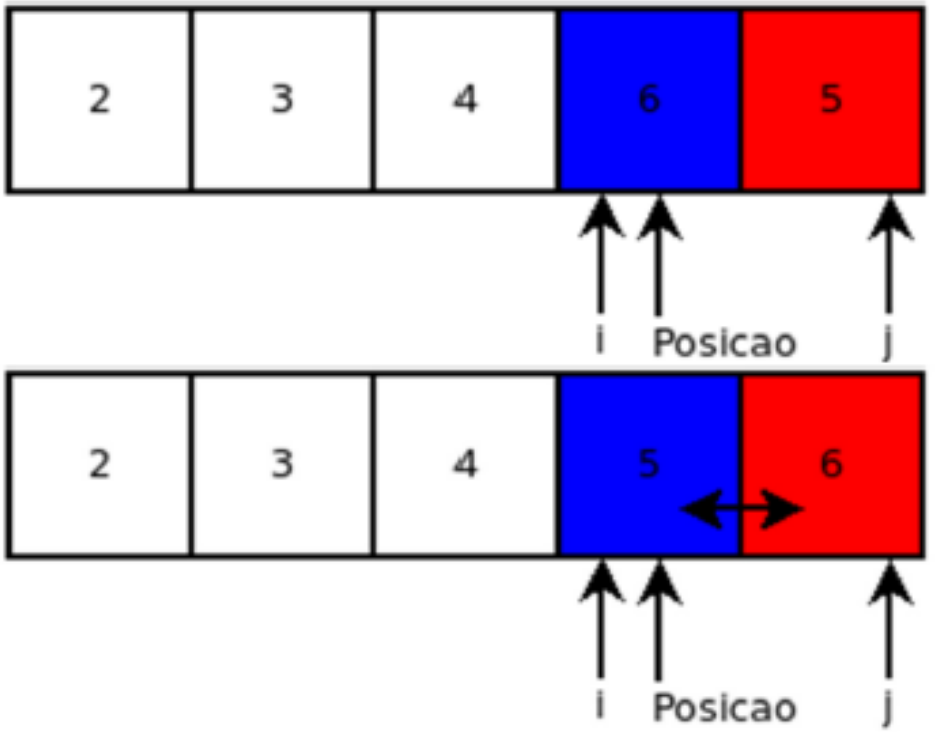
```
Para i de 0 até lista.tamanho -1
    posicao = i
    Para j de i+1 até lista.tamanho -1
        Se o lista[j] < lista[posicao]
            posicao = j
    troca(lista[posicao], lista[i])
```

Straight Sort



Straight Sort





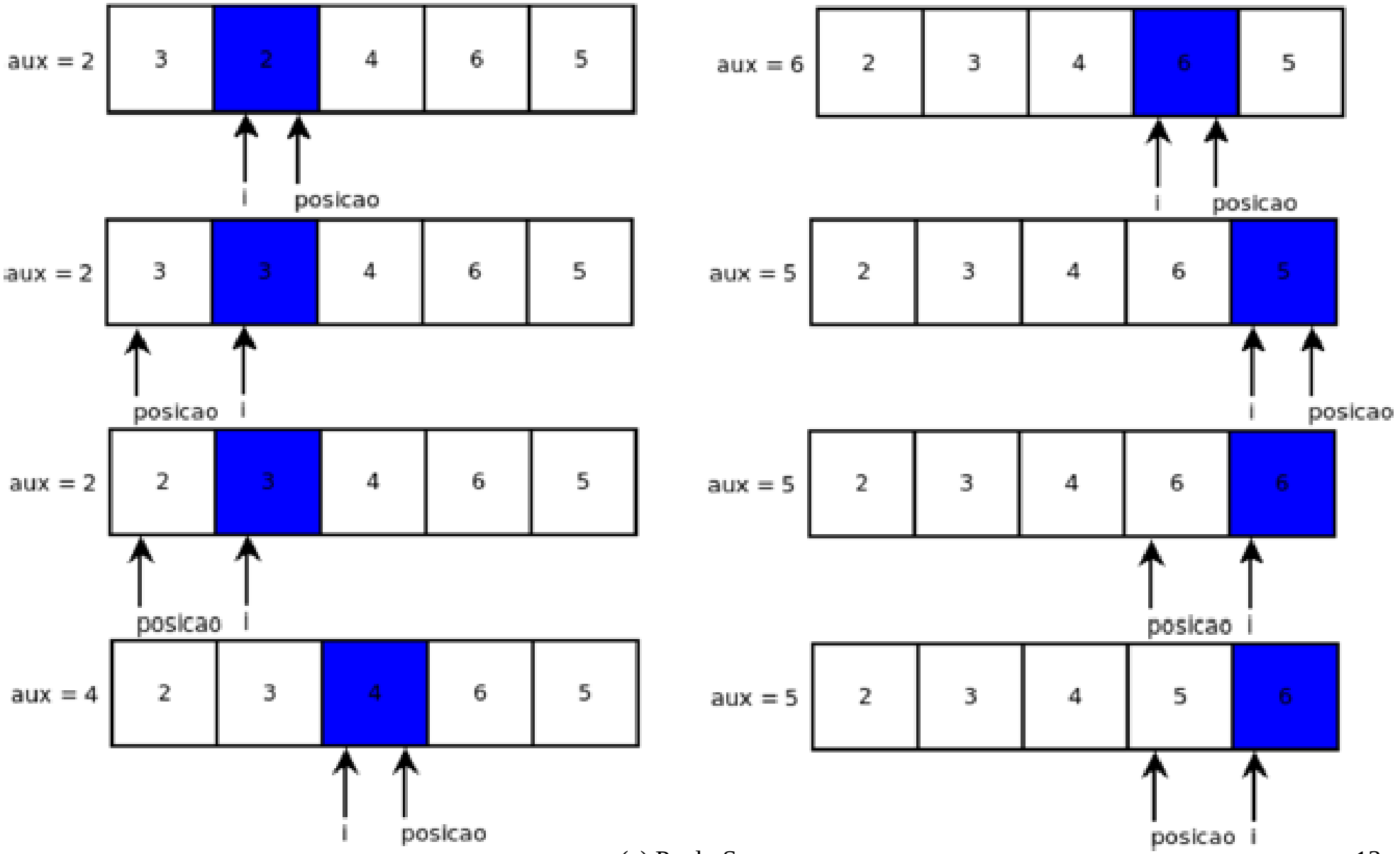
Análise do Straight

- Características
- Muitas comparações ($n^2 / 2$)
- Poucas Trocas (n)
- Complexidade Temporal
- Pior, melhor e caso médio
- $O(n^2)$ - quadrático
- Complexidade Espacial
- S(1) – constante

Insertion Sort

```
Para i de 1 até lista.tamanho -1
    aux = lista[i]
    posicao = i
    Enquanto (posicao >= 1 e aux < lista[posicao-1])
        lista[posicao] = lista[posicao-1]
        posicao = posicao-1
     lista[posicao] = aux
```

Insertion Sort



Análise do Insertion

- Características
- Faz atribuições (1 instrução) e não trocas (3 instruções)
- Poucas Trocas (n)
- Complexidade Temporal
- Pior caso e caso médio
  - $O(n^2)$ - quadrático
- melhor
  - O(n) - linear
- Complexidade Espacial
- S(1) – constante

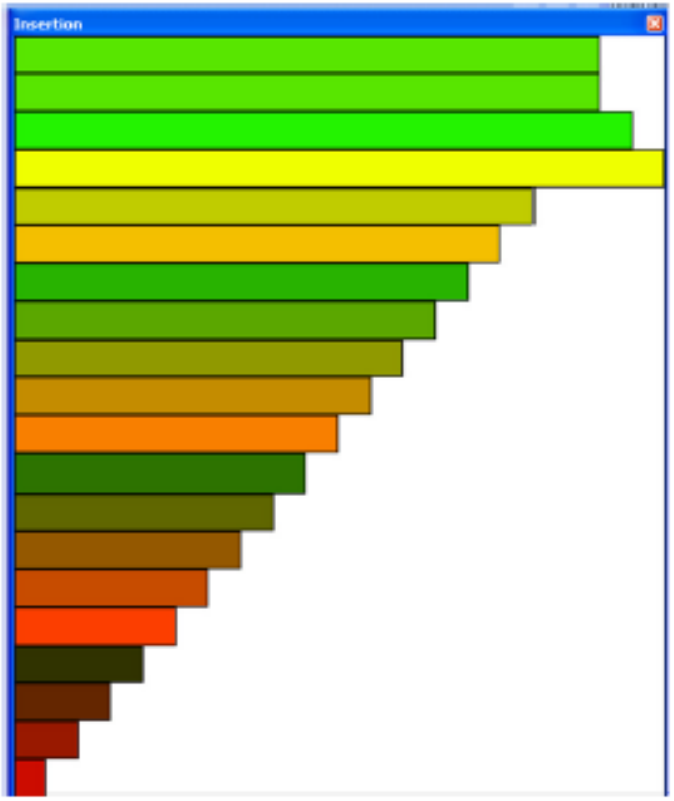
Ordenação simples

- Algoritmos
- Bubble Sort
- Straight Sort
- Insertion Sort

- Características
- Complexidade temporal
  - Dois ciclos
  - Sem divisão do espaço
  - O(n^2)
- Complexidade espaçial
  - Variáveis auxiliares
  - S(1)
- Simples de programar
- Adequados para pequenas quantidades de dados

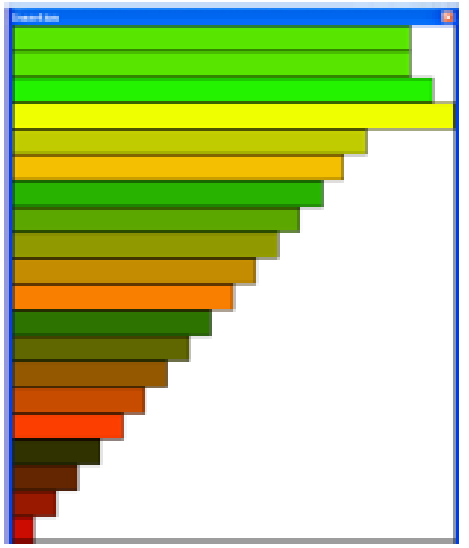
Shell Sort

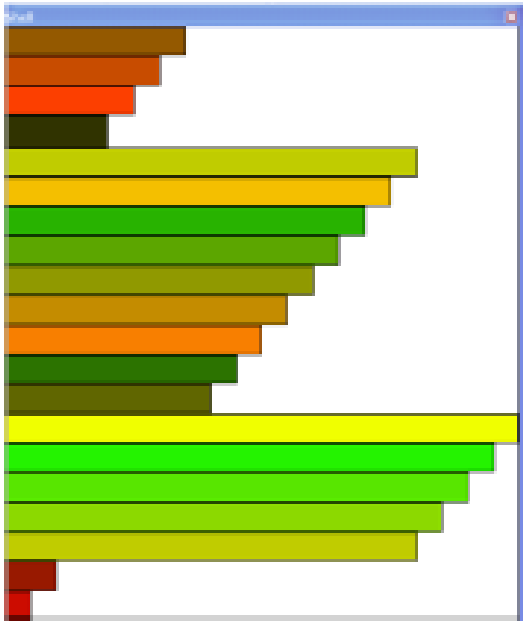
- Descoberto em 1959 por Donal Shell
- Tem complexidade subquadrática
- Inferior a o(n^2)
- Baseado no Insertion
- Insertion
  - Insertion troca elementos seguidos
  - Eficiente quando os elementos estão quase ordenados
    - Poucos deslocamentos
  - Ineficiente quando os elementos estão muito desordenados ou ordenados por ordem inversa
    - Deslocações de grandes quantidades de dados



Shell Sort

- Algoritmo
- Troca elementos espaçados por uma métrica
  - Começa por trocar elementos que estão afastados
  - Termina com um espaçamento unitário (Insertion)
  - O espaçamento (h) é dado por uma função
- Original - Divisão por dois
  - 100 50 25 12 6 3 1
- Outras funções
  - **$h_n = (h_{n-1} - 1) / 3$**
  - **$h_n = (5 * h_{n-1}) / 11$**
  - **$h_n = h_{n-1} / 2.2$**
  - **Números primos**

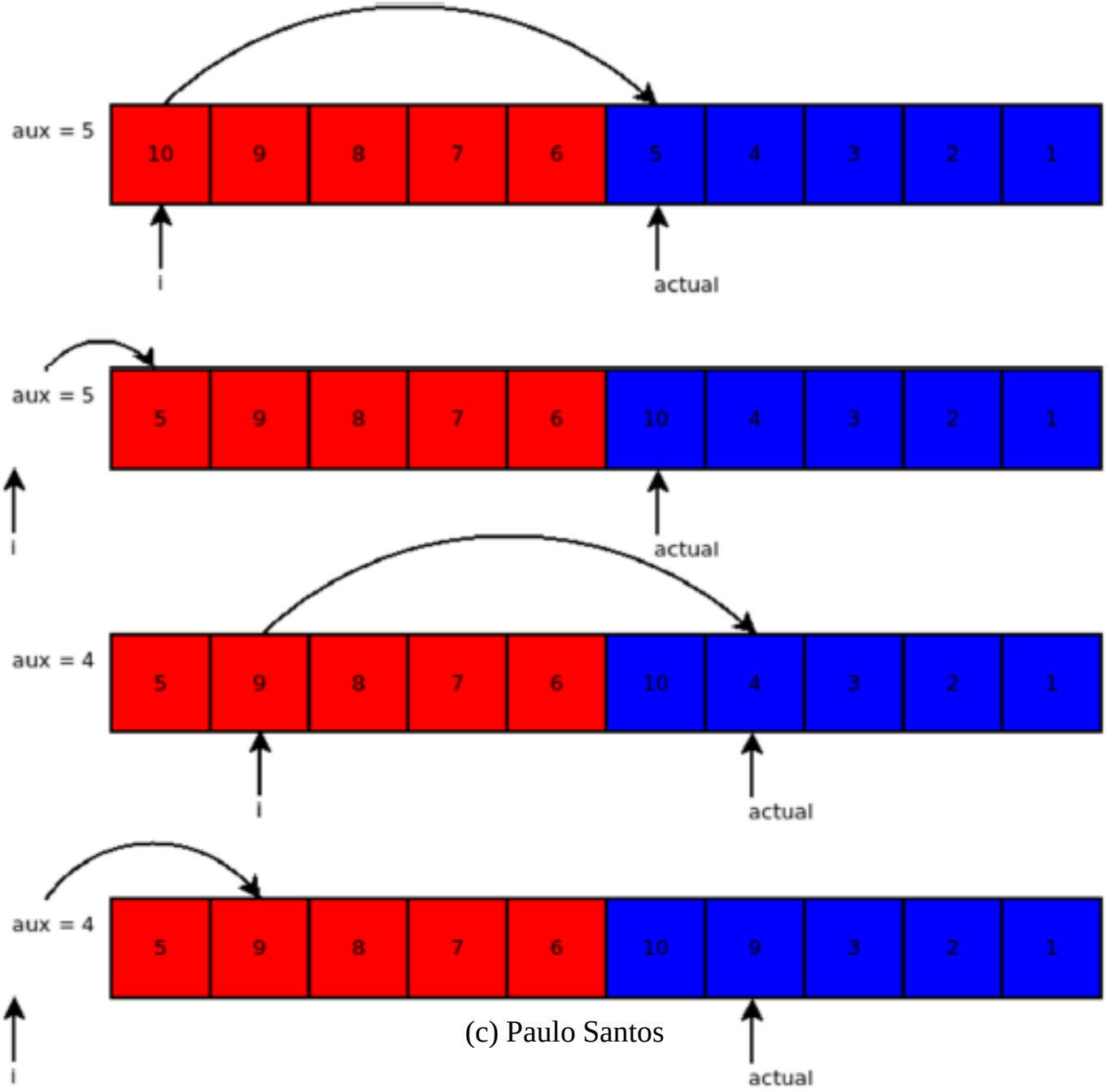




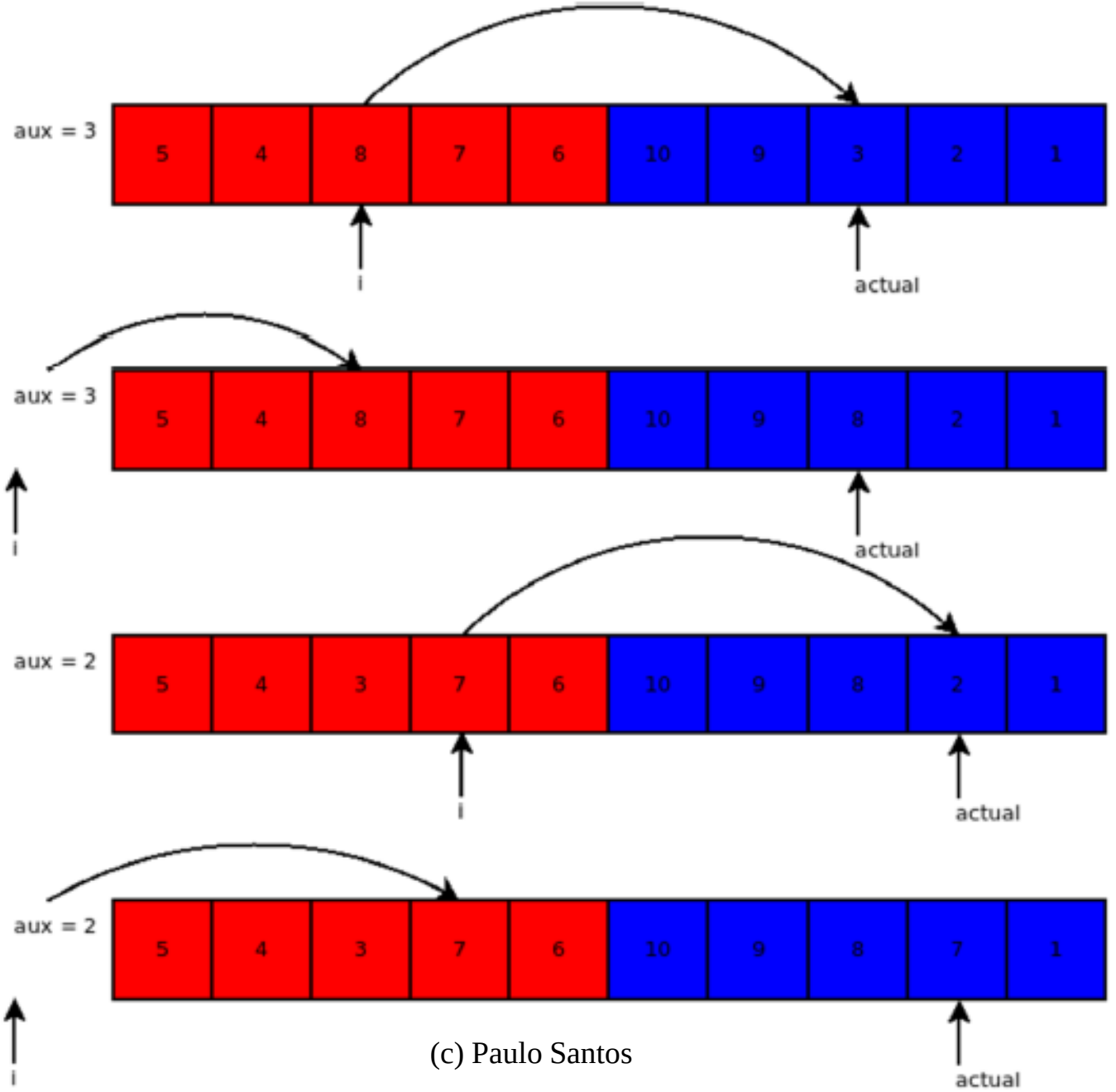
Shell Sort

```
incremento = tamanho / 2;
enquanto incremento <> 0
    actual = incremento
    enquanto actual < tamanho
        aux = lista[actual]
        i = actual – incremento
        enquanto i >= 0 e aux < lista[i]
            lista[i+incremento] = lista[i]
            i = i – incremento
        lista[i+incremento] = aux
        actual = actual + 1
    incremento = incremento / 2
```

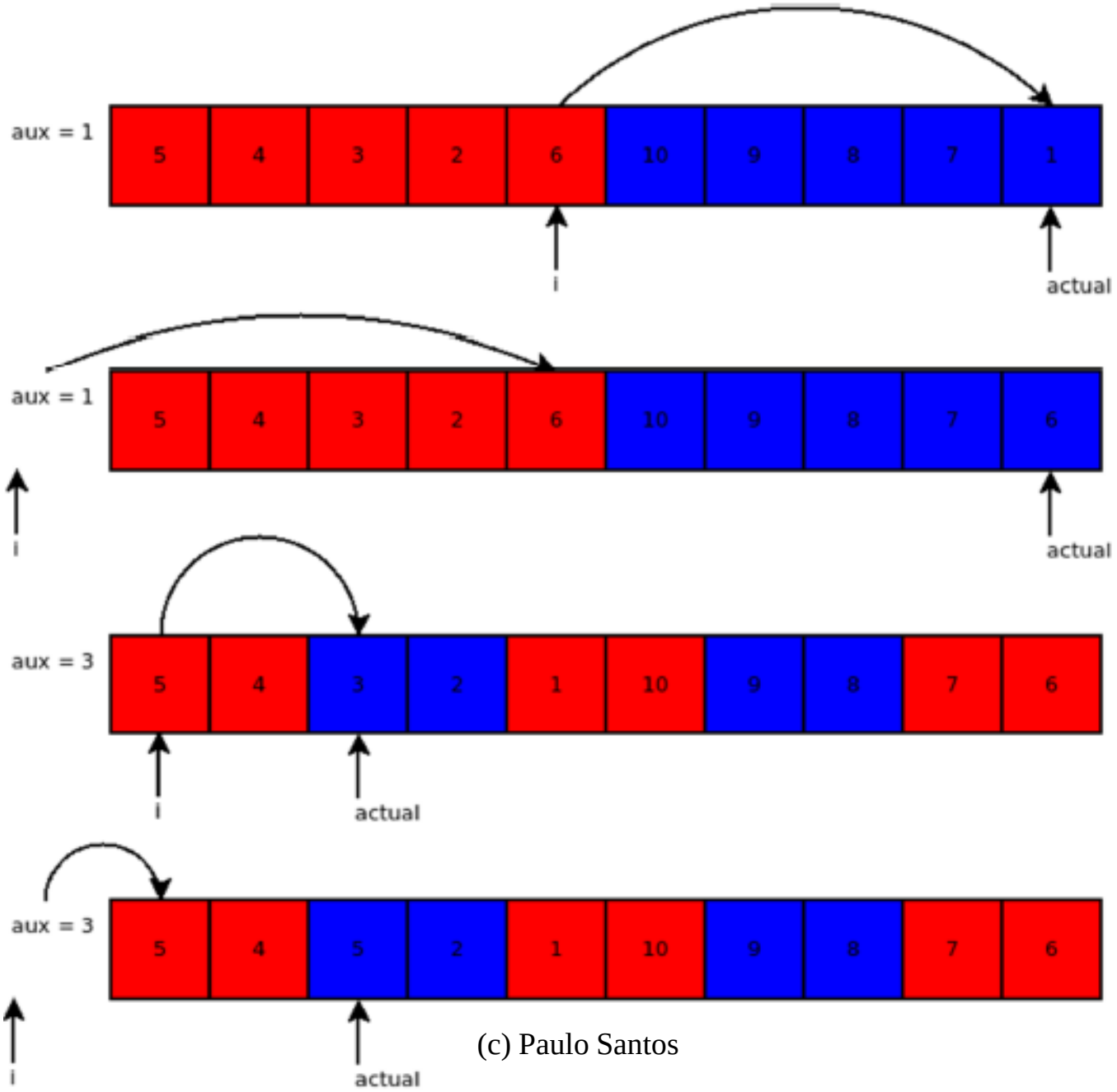
Shell Sort


aux = 5
10 9 8 7 6 5 4 3 2 1
i
actual
aux = 5
5 9 8 7 6 10 4 3 2 1
i
actual
aux = 4
5 9 8 7 6 10 4 3 2 1
i
actual
aux = 4
5 9 8 7 6 10 9 3 2 1
i
actual

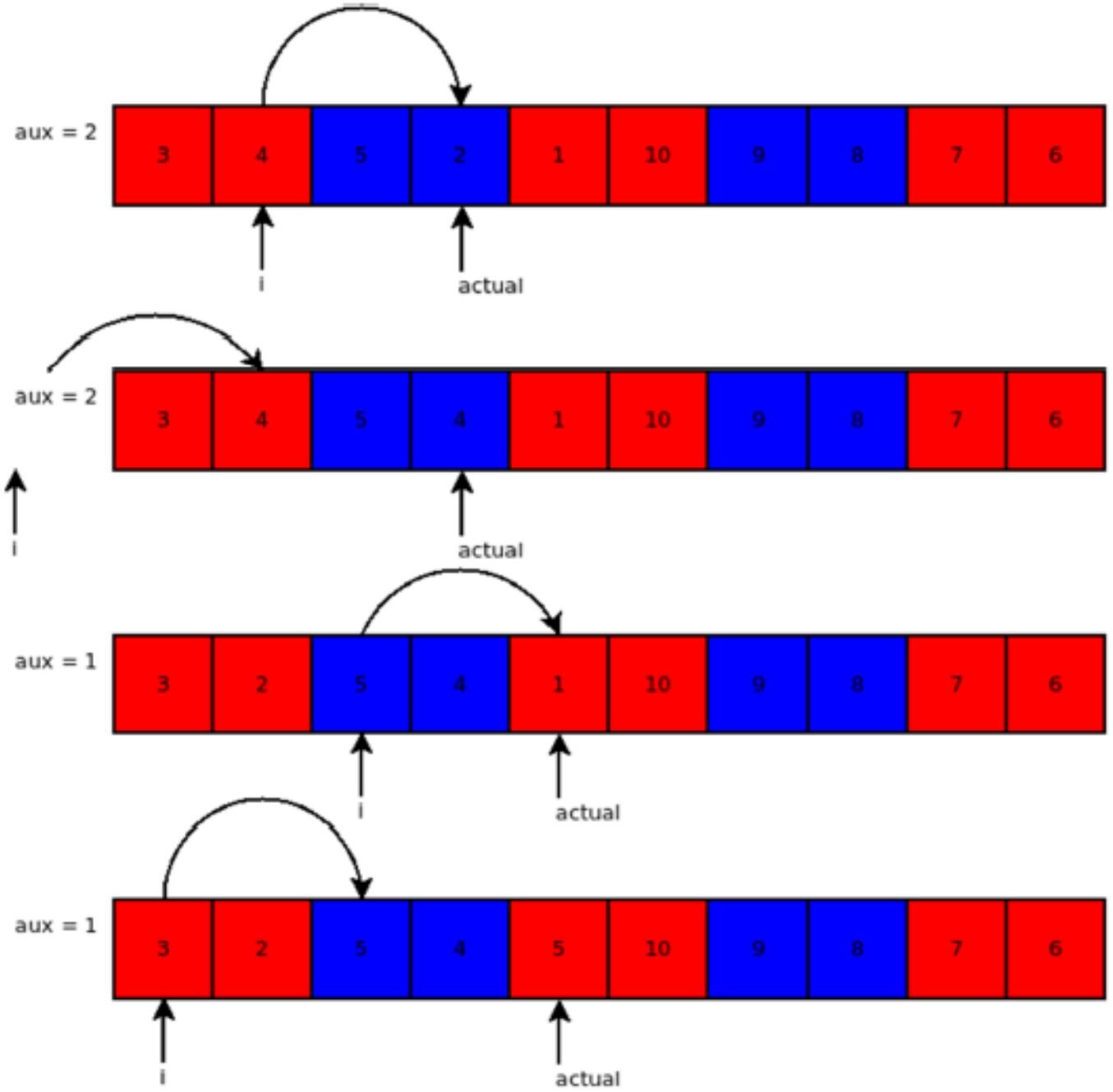
Shell Sort


aux = 3
5 4 8 7 6 10 9 3 2 1
i
actual
aux = 3
5 4 8 7 6 10 9 8 2 1
i
actual
aux = 2
5 4 3 7 6 10 9 8 2 1
i
actual
aux = 2
5 4 3 7 6 10 9 8 7 1
i
actual

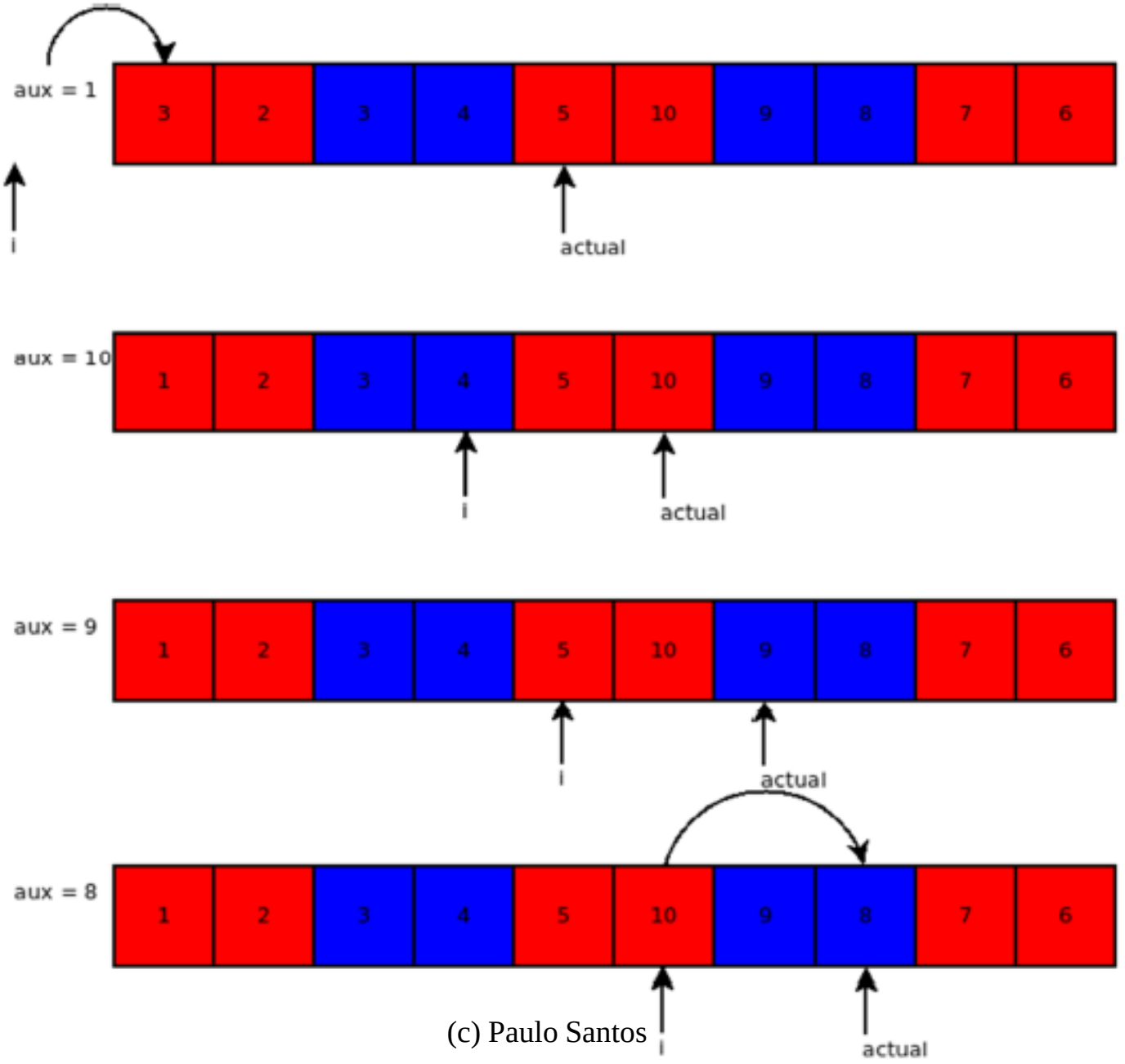
Shell Sort


aux = 1
5 4 3 2 6 10 9 8 7 1
i
actual
aux = 1
5 4 3 2 6 10 9 8 7 6
i
actual
aux = 3
5 4 3 2 1 10 9 8 7 6
i
actual
aux = 3
5 4 5 2 1 10 9 8 7 6
i
actual

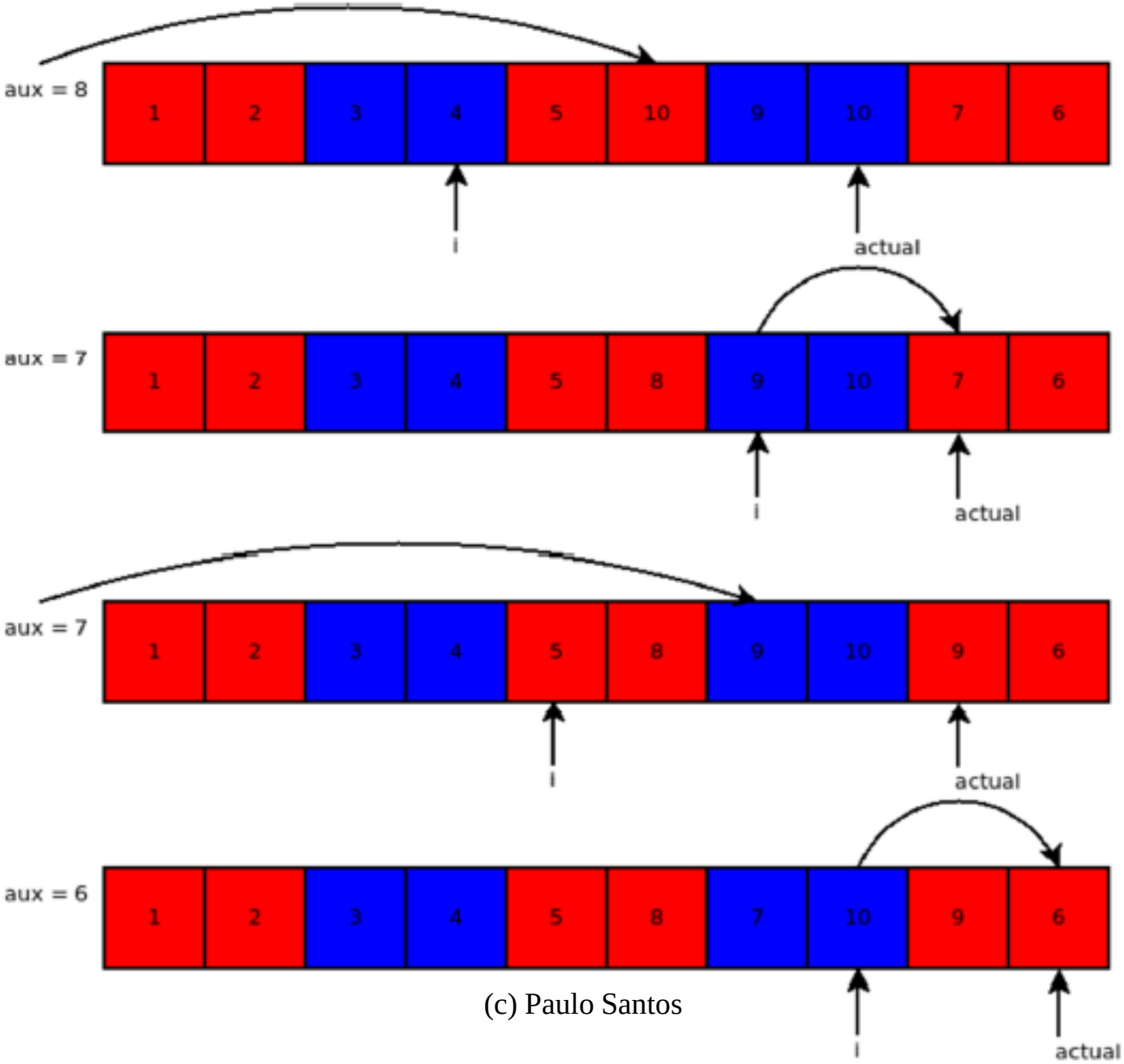
Shell Sort


aux = 2
3 4 5 2 1 10 9 8 7 6
i
actual
aux = 2
3 4 5 4 1 10 9 8 7 6
i
actual
aux = 1
3 2 5 4 1 10 9 8 7 6
i
actual
aux = 1
3 2 5 4 5 10 9 8 7 6
i
actual

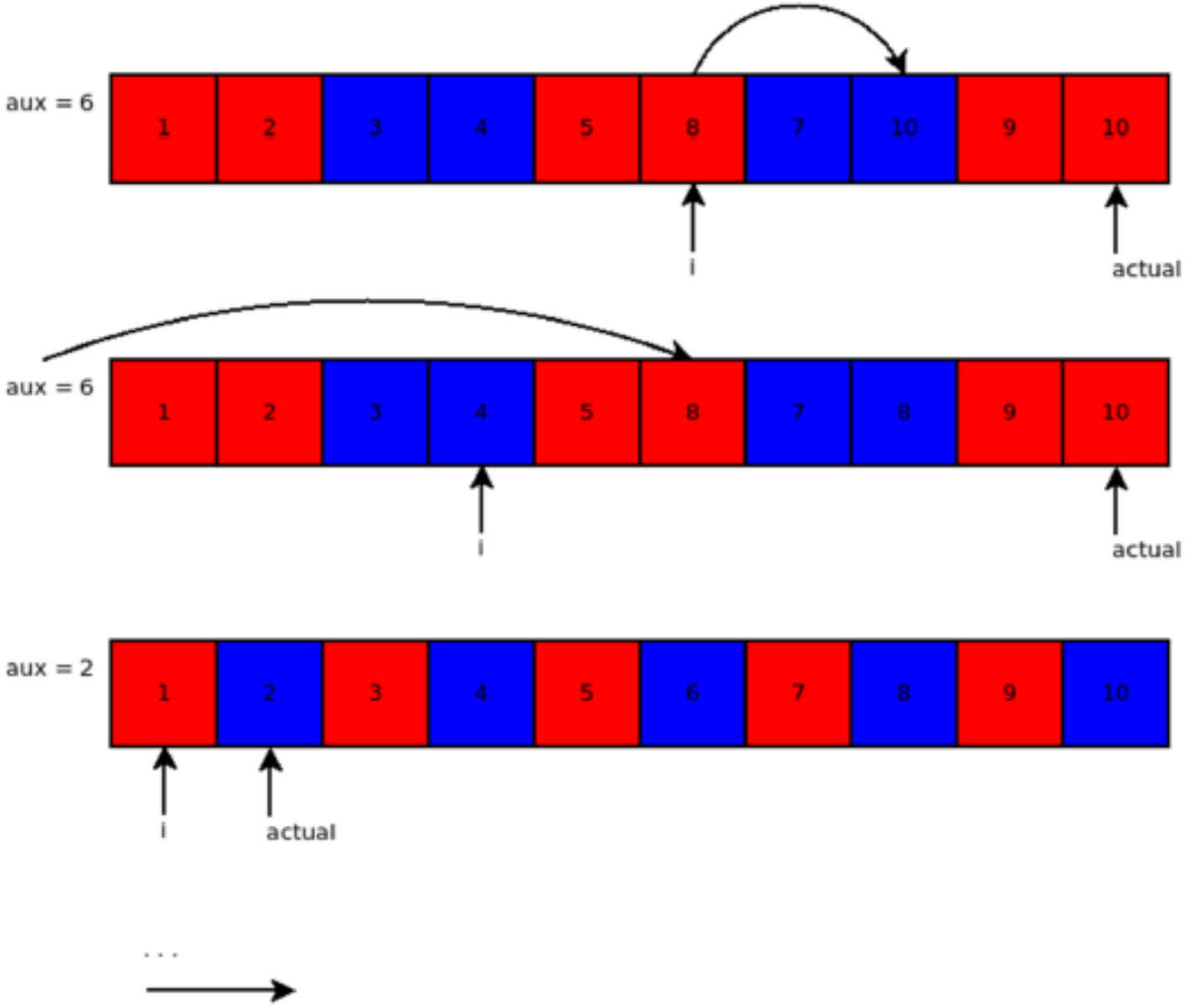
Shell Sort


aux = 1
3 2 3 4 5 10 9 8 7 6
i
actual
aux = 10
1 2 3 4 5 10 9 8 7 6
i
actual
aux = 9
1 2 3 4 5 10 9 8 7 6
i
actual
aux = 8
1 2 3 4 5 10 9 8 7 6
i
actual

Shell Sort

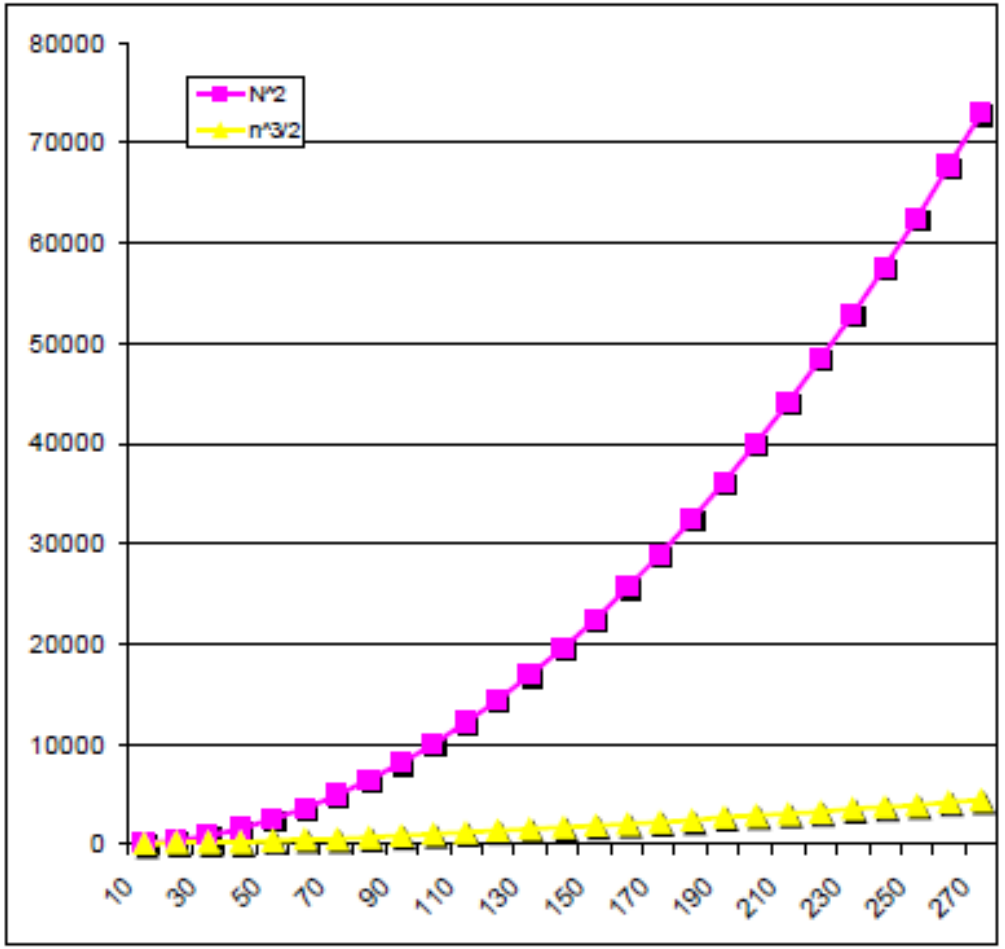

aux = 8
1
2
3
4
5
10
9
10
7
6
i
actual
aux = 7
1
2
3
4
5
8
9
10
7
6
i
actual
aux = 7
1
2
3
4
5
8
9
10
9
6
i
actual
aux = 6
1
2
3
4
5
8
7
10
9
6
i
actual

Shell Sort


aux = 6
1 2 3 4 5 8 7 10 9 10
i
actual
aux = 6
1 2 3 4 5 8 7 8 9 10
i
actual
aux = 2
1 2 3 4 5 6 7 8 9 10
i
actual
. . .

Análise do Shell

- Complexidade temporal
- Experimentalmente
- Pior caso
  - $O(n^{3/2})$
- Melhor caso
  - $O(n^{7/6})$
  - Inferior ao quadrático



- Complexidade Espacial
- S(1) – constante
- Adequado para quantidades moderadas de dados

Merge Sort

- A Ordenação de um vector com dois elementos faz-se em tempo constante
- A fusão de dois vectores ordenados faz-se em tempo Linear
- Algoritmo
- Dividir o vector ao meio até que cada sub-vector tenha no máximo dois elementos
  - Ordenar os sub-vectores
  - Juntar os sub-vectores

Merge Sort

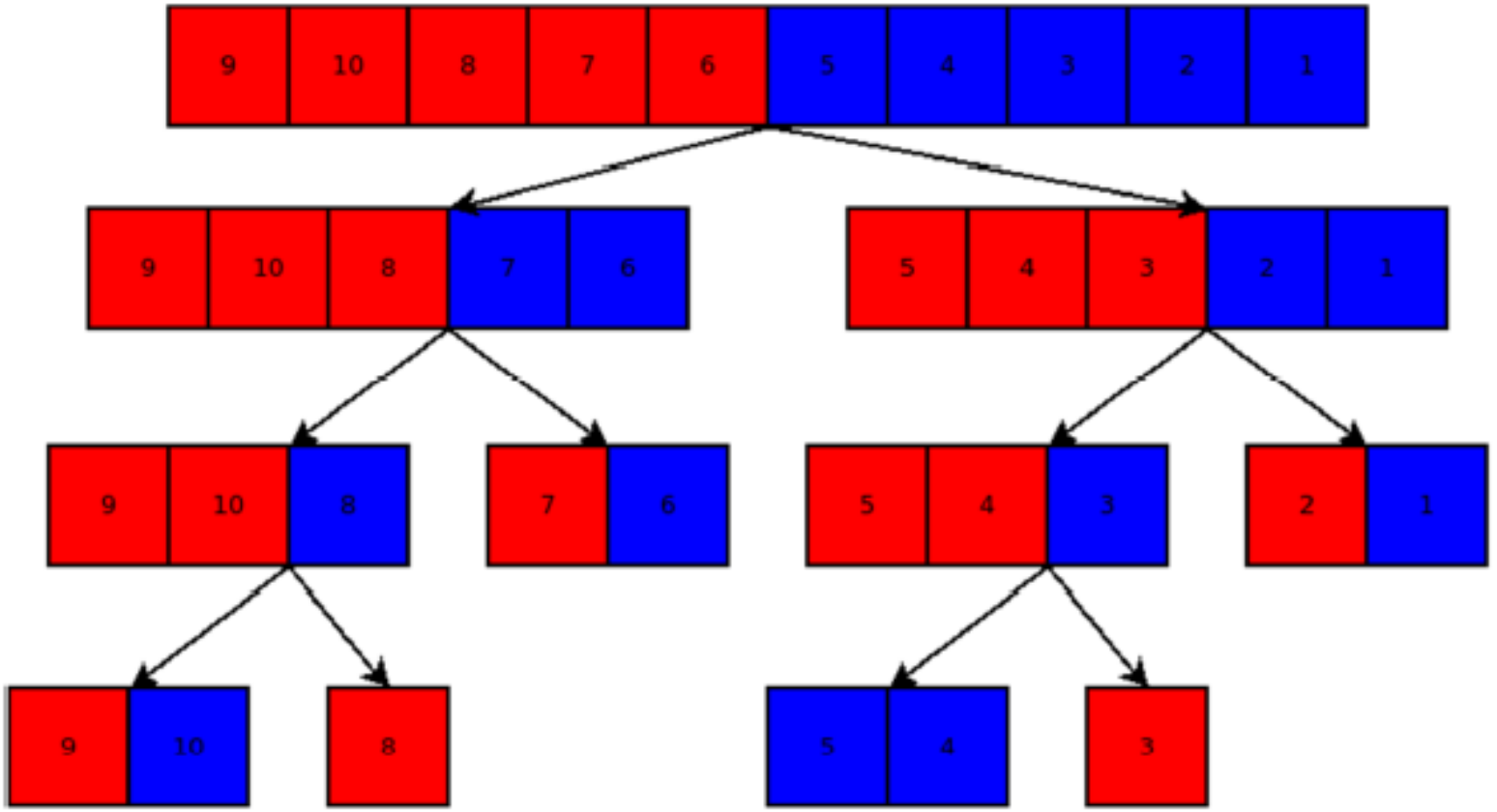
```
Object[] lista = ..... //Lista a ordenar
Object[] tmp = new Object[lista.length];
mergeSort(lista, tmp, 0, length-1);

void mergeSort(Object[] lista, Object[] tmp, int esquerda, int
    direita) {
        Se  direita – esquerda > 1 // tem mais de 2 elementos
            meio = (esquerda + direita) / 2
            mergeSort(lista, tmp, esquerda, meio)
            mergeSort(lista, tmp, meio + 1, direita;
            merge(lista, tmp, esquerda, direita)
        Senão se lista[direita] < lista[esquerda]
            troca(esquerda, direita)
}
```

Merge Sort

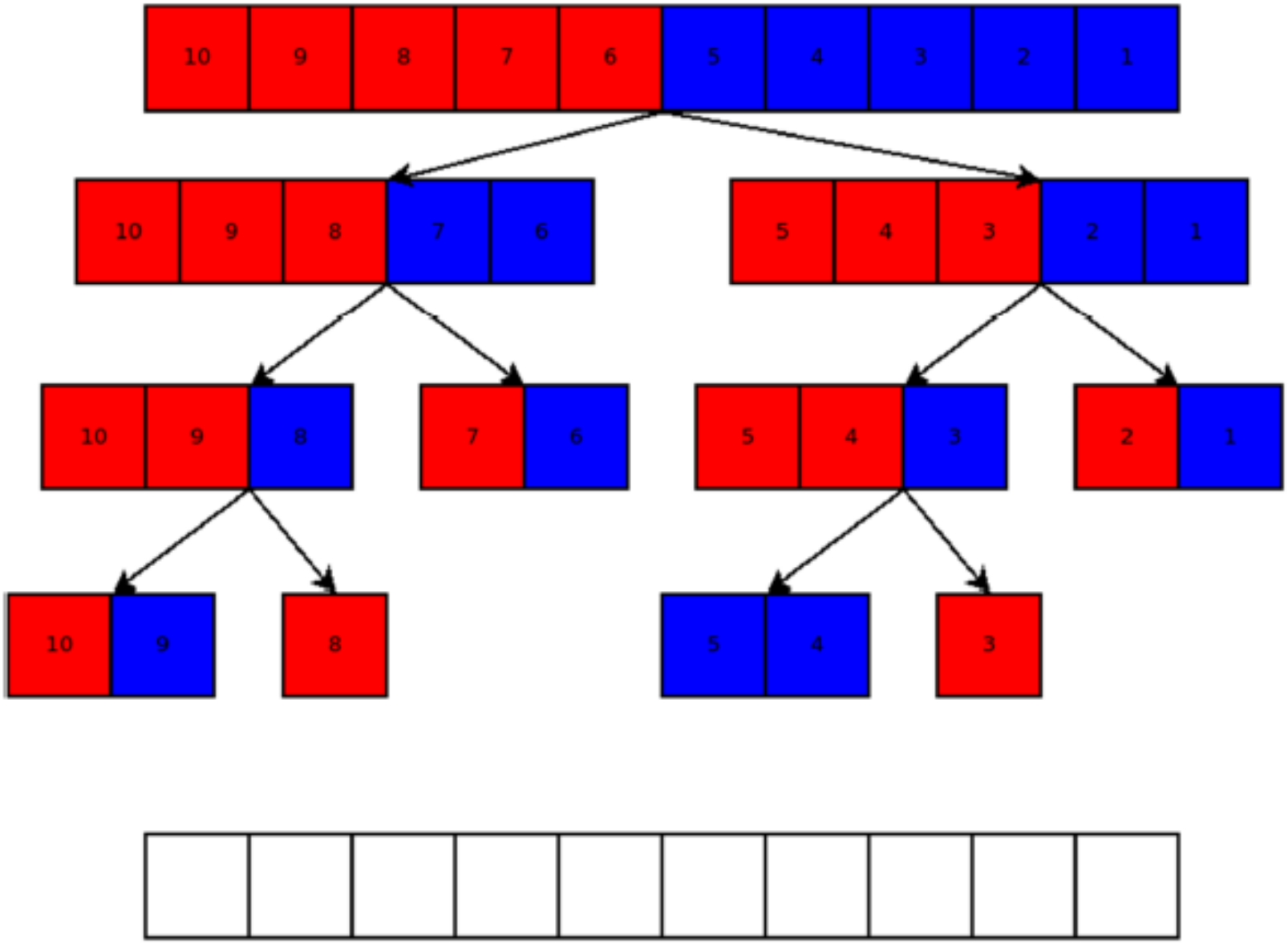
```
void merge(Object[] lista, Object[] tmp,
                int esquerda, int direita) {
    meio = (esquerda + direita) / 2
    posEsquerda = meio
    posDireita = meio + 1
    posTmp = esquerda
    nElementos = direita - esquerda + 1
    // começa a copia
    Enquanto esquerda <= meio e
                posDireita <= direita
        Se lista[esquerda] < listao[posDireita]
            tmp[posTmp] = lista[esquerda]
            esquerda = esquerda + 1
        Se não
            tmp[posTmp] = lista[posDireita]
            direita = direita + 1
        posTmp = posTmp + 1
    // copia o resto da primeira metade
    Enquanto esquerda <= posEsquerda
        tmp[posTmp] = lista[esquerda]
        posTmp = posTmp + 1
        esquerda = esquerda + 1
    // copia o resto da segunda metade
    Enquanto posDireita <= direita
        tmp[posTmp] = lista[posDireita]
        posTmp = posTmp + 1
        posDireita = posDireita + 1
    // copia do array temporario para o original
    Para i de 0 até nElementos
       lista[direita] = tmp[direita]
       direita = direita - 1
```

Merge Sort

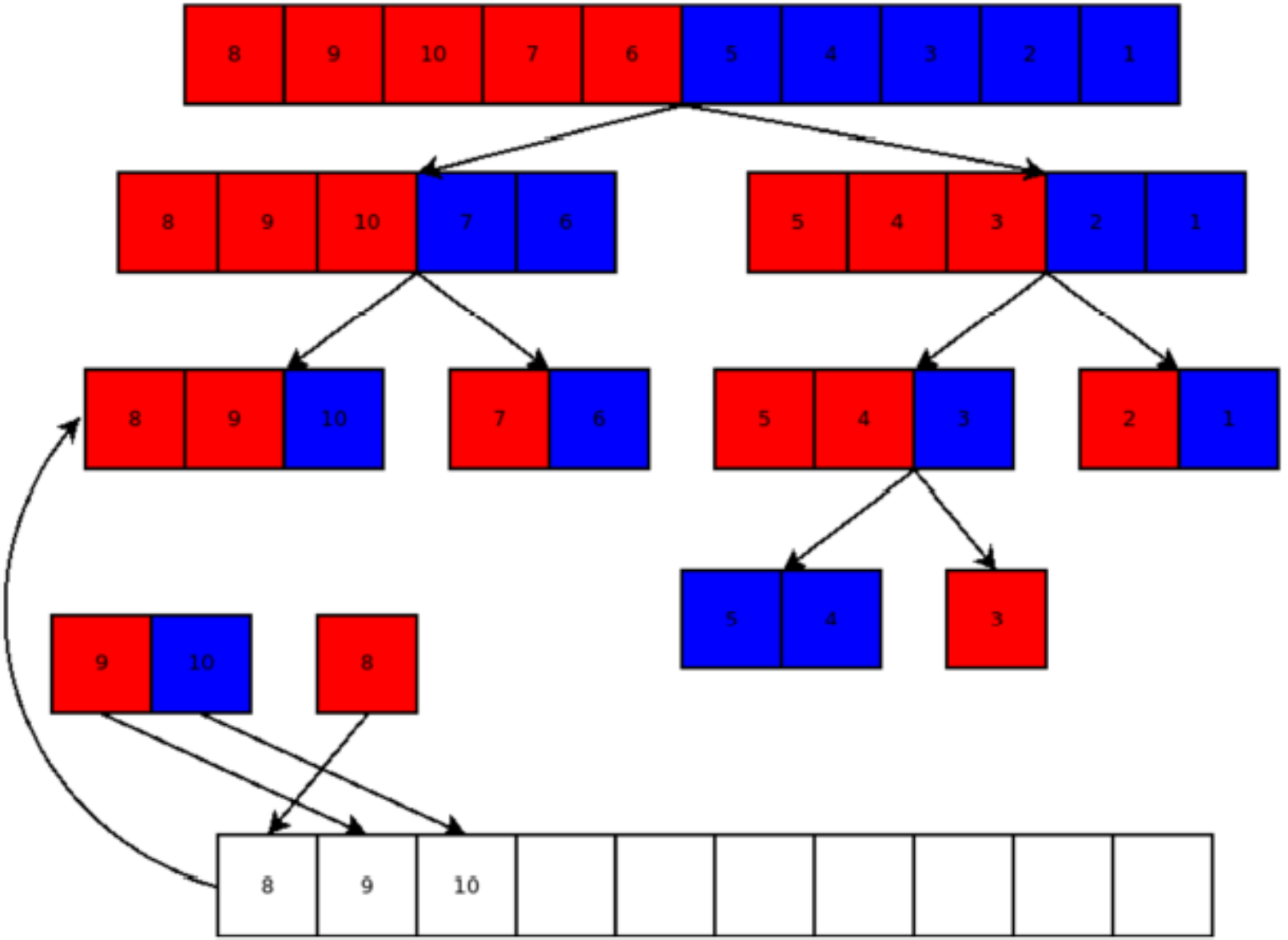




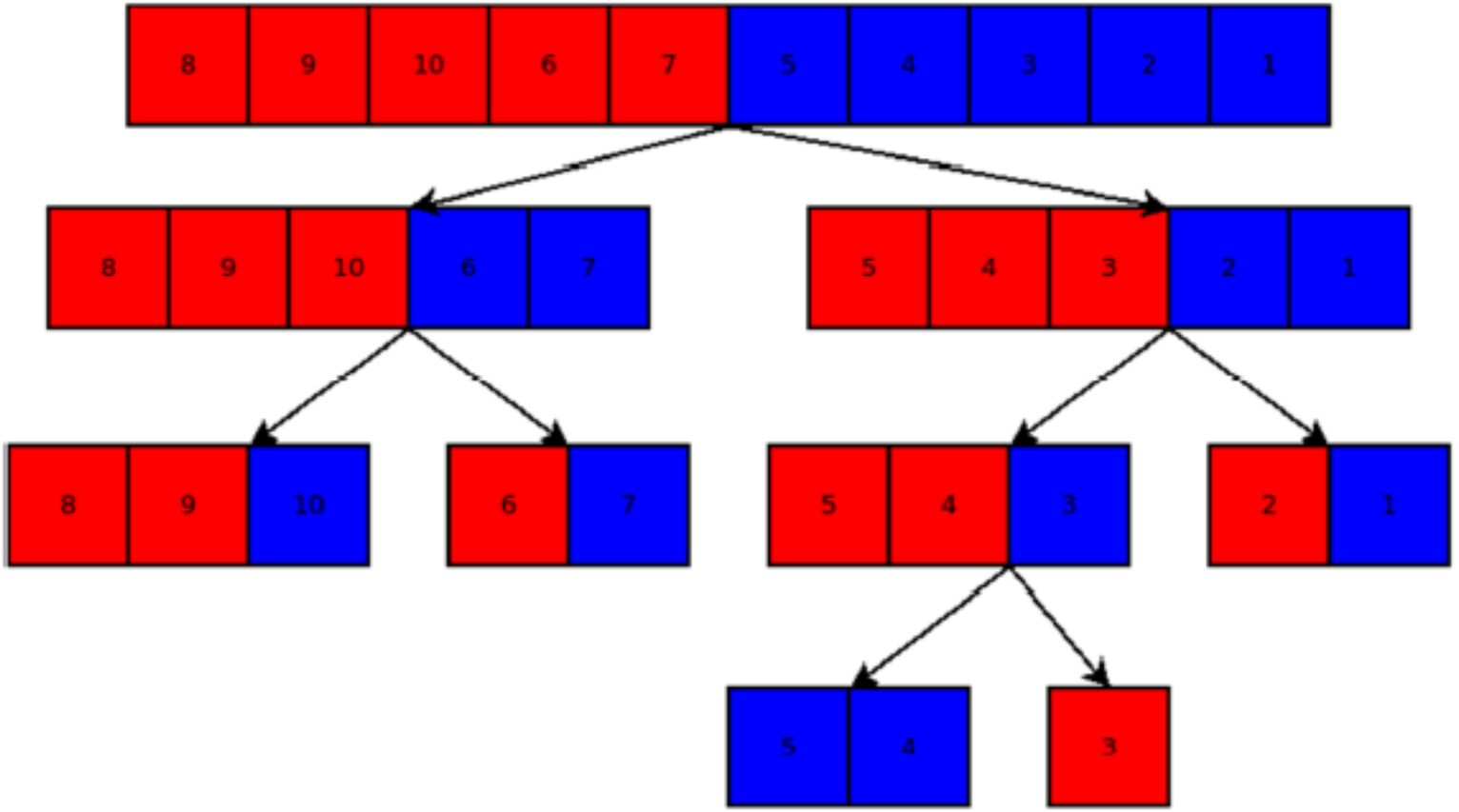
Merge Sort



Merge Sort

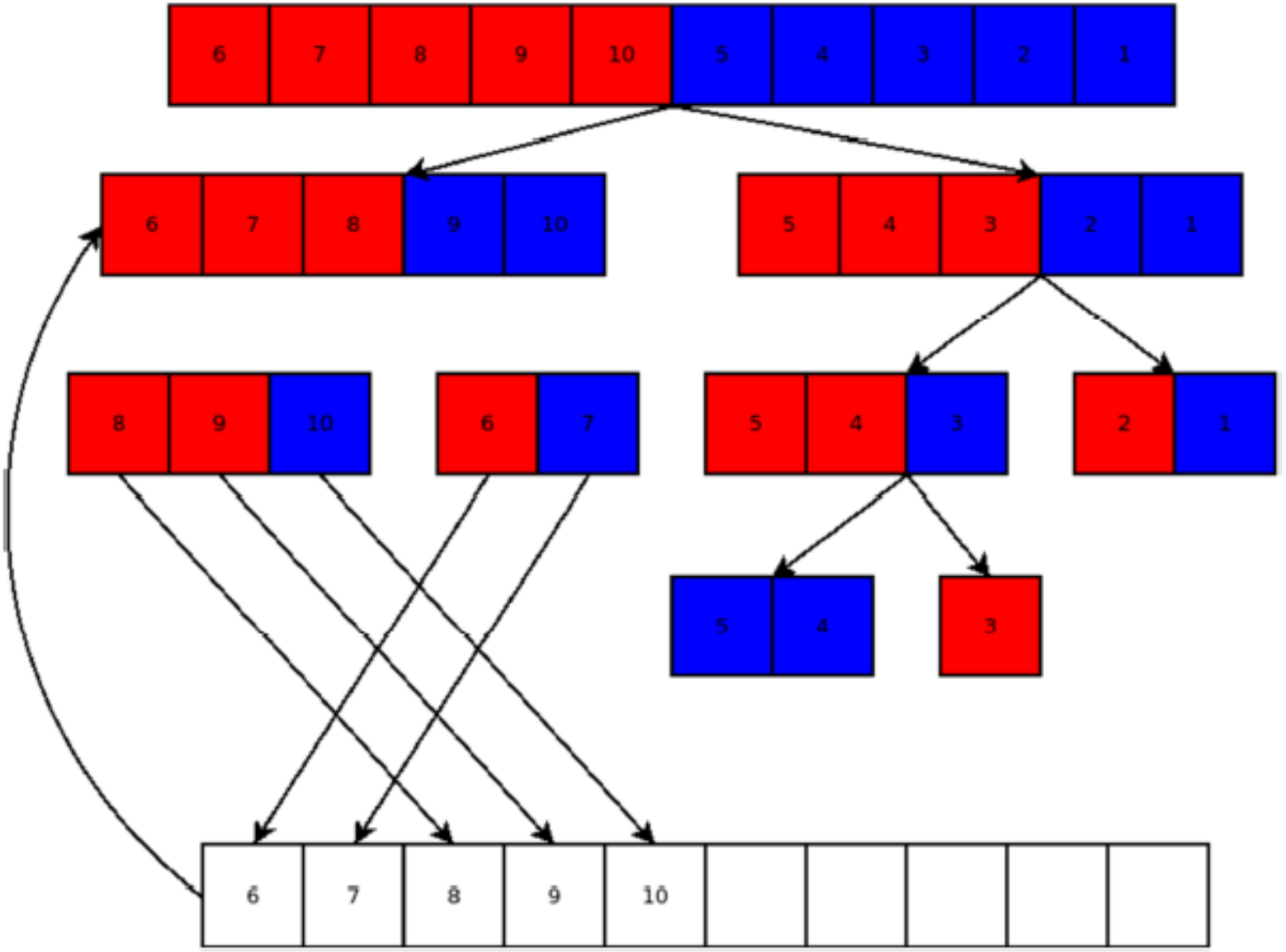


Merge Sort

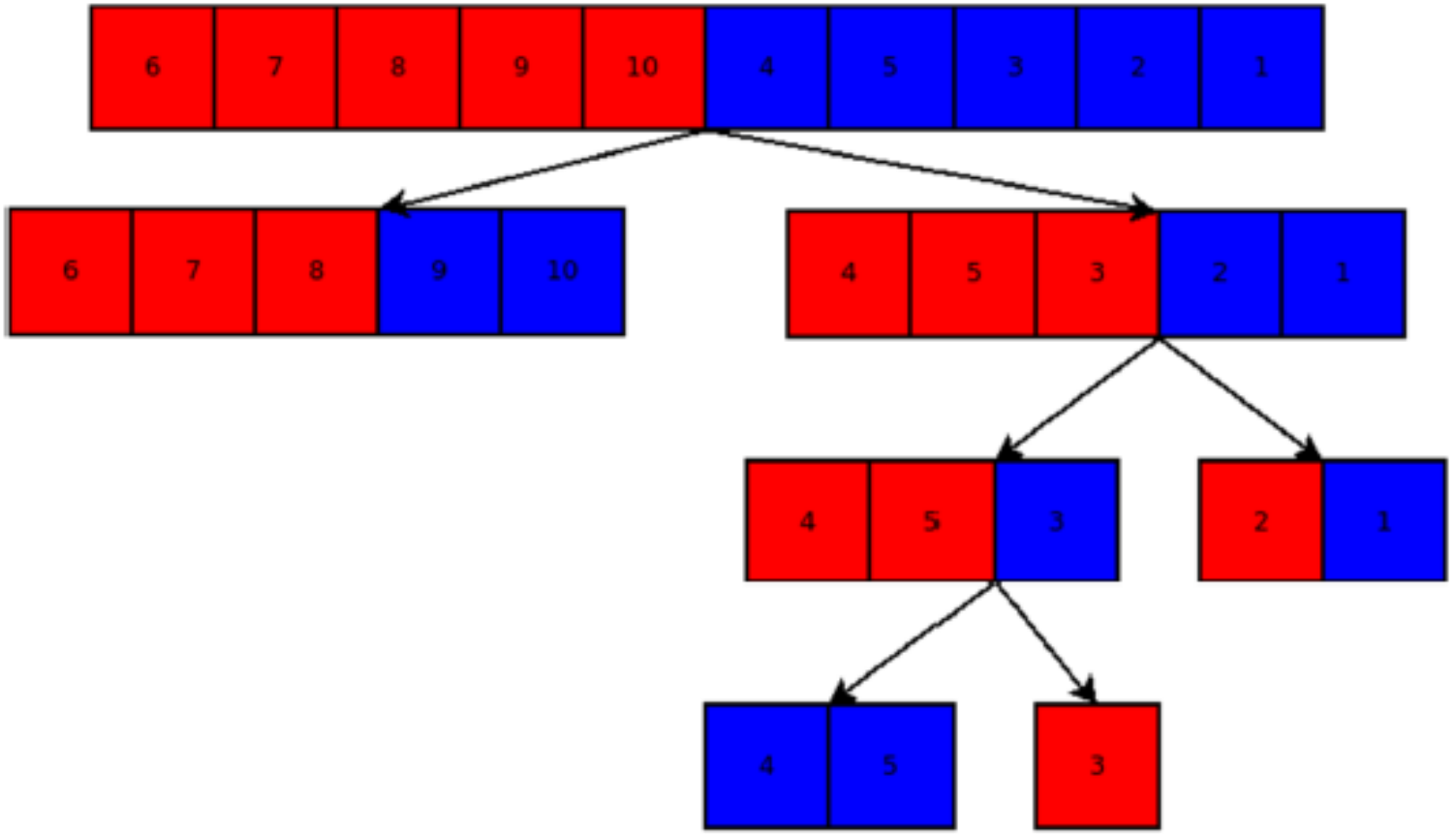




Merge Sort

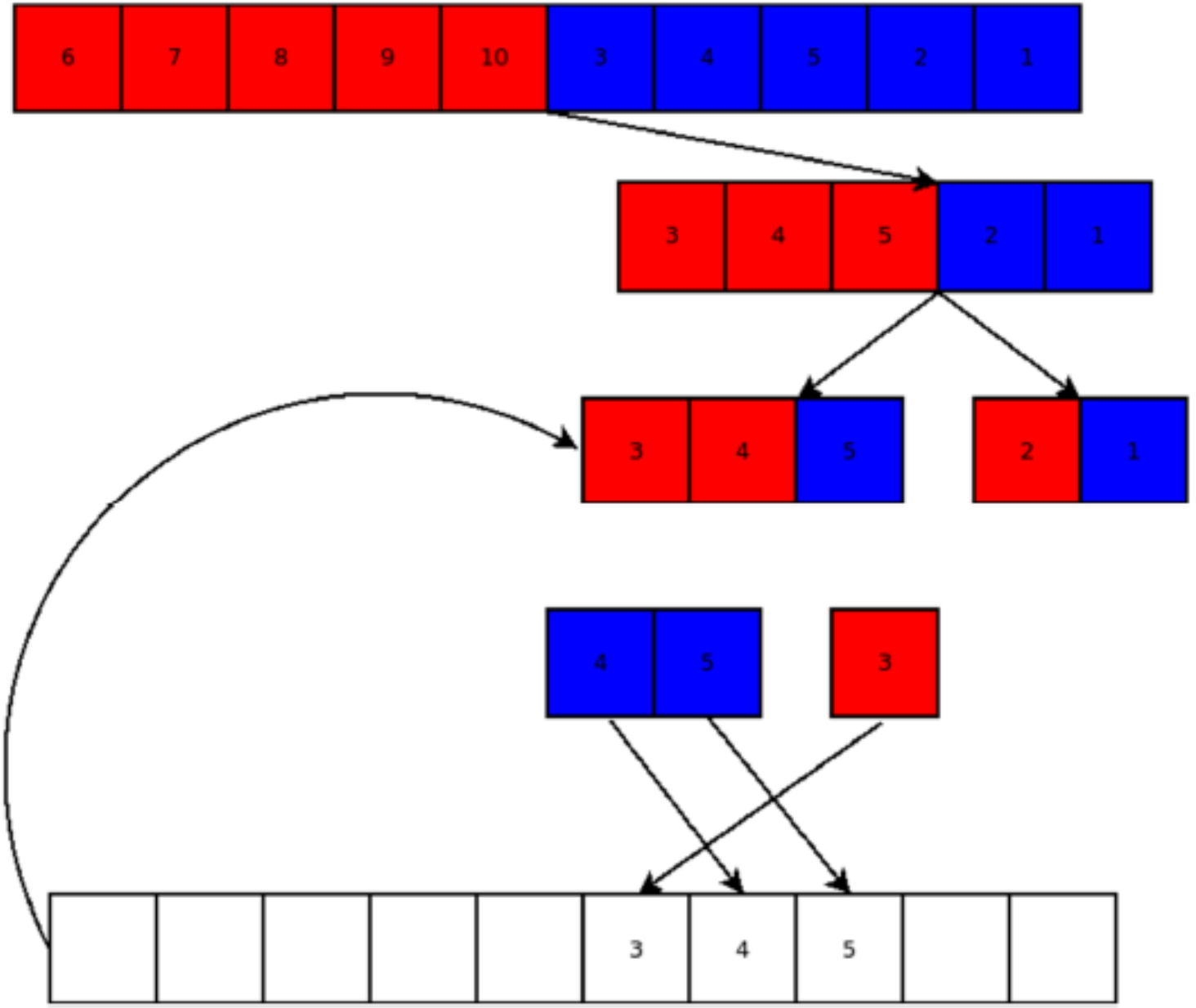


Merge Sort

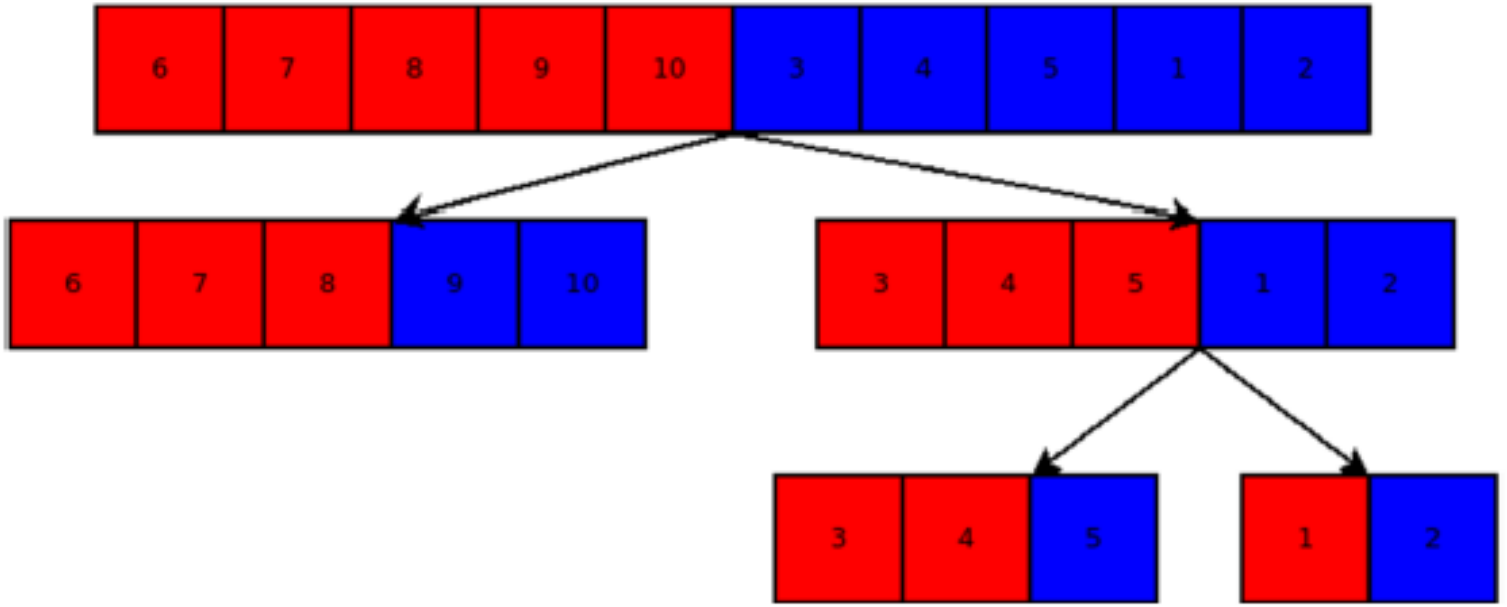




Merge Sort

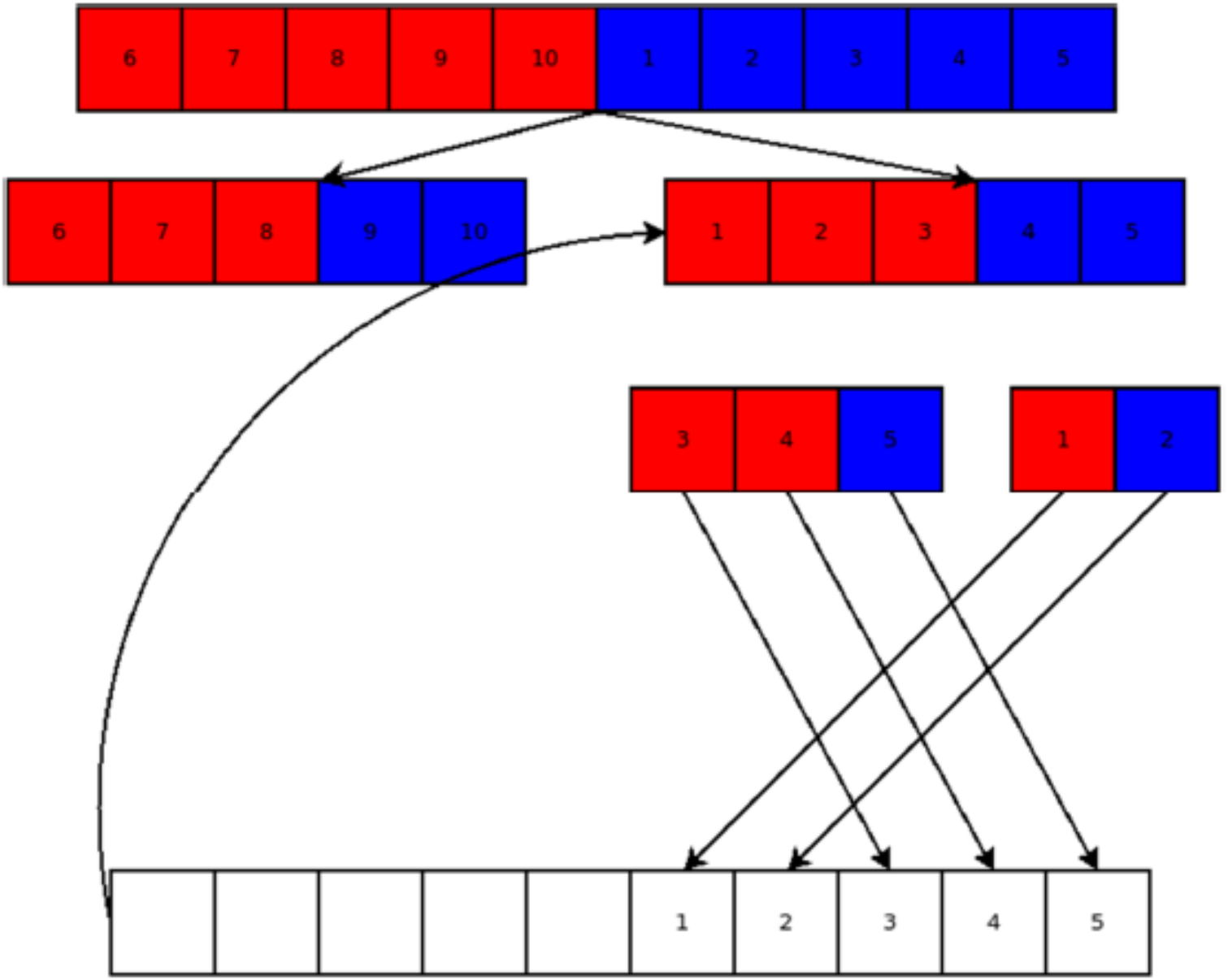


Merge Sort

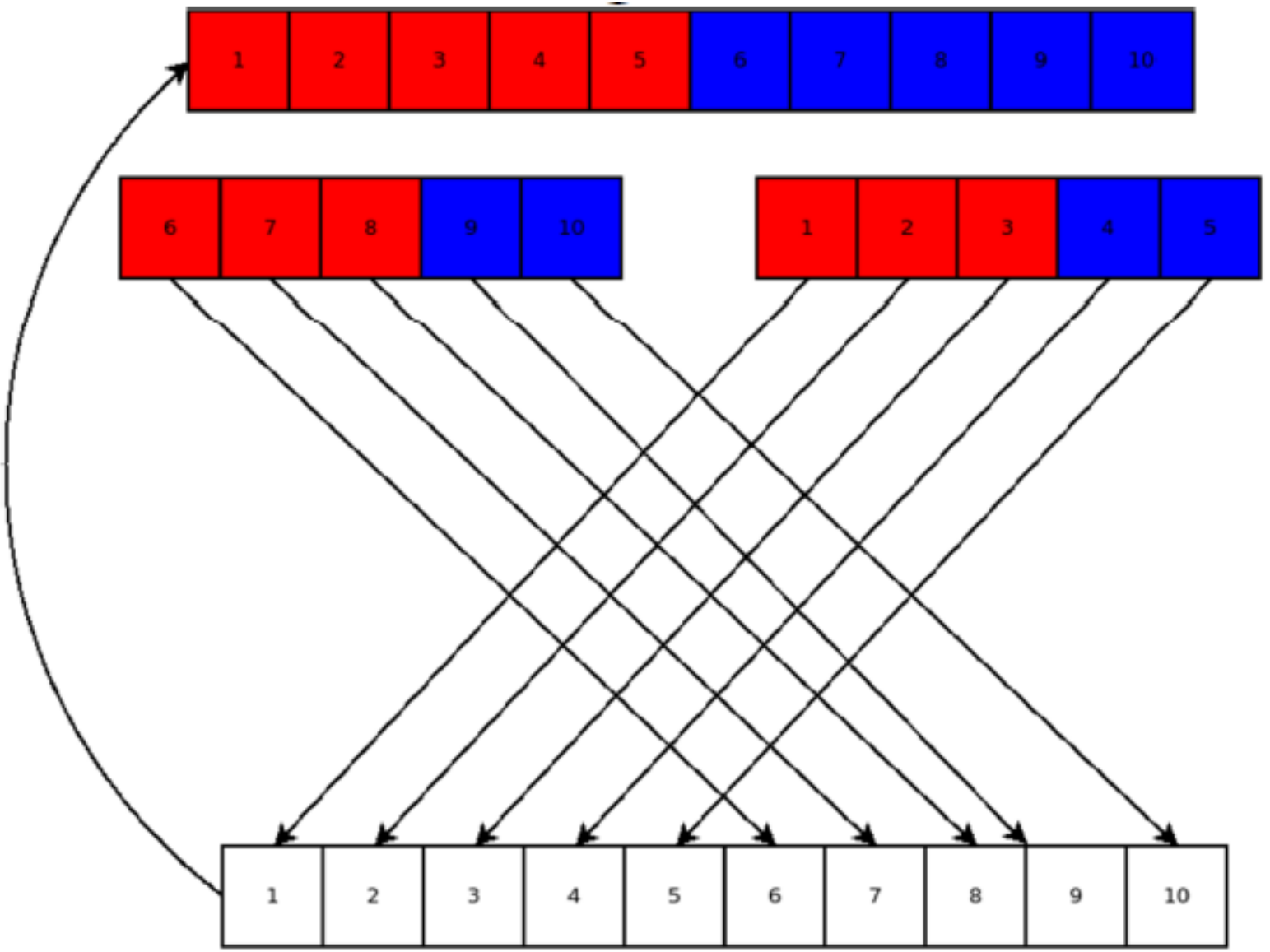




Merge Sort



Merge Sort



Análise do Merge

- Complexidade temporal
  - Log_2 n partições
  - N instruções para as juntar
  - O(nlog(n))
- Complexidade Espacial
- Partições recursivas
  - Log_2
- Array auxiliar de n elementos
- S(n) + S(Log_2) = S(n)

